CHAPTER 1

DESIGNING A LIBRARY MANAGEMENT SYSTEM

1 Getting Ready: Library Management System 2
2 Expectations from the interviewee 2
3 Design approach 4
4 Requirements for the Library Management System 4
5 Requirement collection 5
6 Use Case Diagram for the Library Management System . 6
7 Class Diagram for the Library Management System . 11
8 Sequence Diagram for the Library Management System . 25
9 Activity Diagram for the Library Management System . 27
10 Code of Library Management System 31
11 Summary: Designing a Library Management System . 41
12 Key Insights and Tips: Designing a Library Management System 45

This chapter teaches you the design of a library management system with requirements, use case, class, sequence, activity diagrams, and implementation code.

1 Getting Ready: Library Management System

A **library management system (LMS)** aims to automate all library activities. It is software that helps manage all the primary functions of library management. With the help of a library management system, we can organize, handle, and maintain the records of numerous books and members comprehensively and systematically.
A **librarian** can use this software to track the number of books in the library and retain several records, including new books, borrowed books with due dates, members who borrowed books, returned books, fines for late returned books, etc. In short, the library management system stores and updates the complete library database.
LMS also supports maintaining the physical library. The user can keep track of a book's position in the library and search for whether or not the book is currently available. Therefore, LMS helps organize and retrieve library data efficiently.
In this LLD interview case study, you'll focus on:

- Efficiently searching and managing books and other resources by various attributes (title, author, category, etc.).
- Handling book borrowing, returning, and reservation workflows.
- Managing user roles (librarian, member) and their permissions.
- Supporting fine calculation and renewal processes for overdue and renewed books.

Note: This system model can be adapted for academic, public, or private libraries and can support various library policies (such as special collections, inter-library loans, or different user types).

Figure 1.1: An example of the library management system

2 Expectations from the interviewee

The LMS has multiple components, each with its specific requirements and constraints. Let 's look at some of the main expectations the interviewer will want to hear you discuss in more detail.

2.1 Efficient searching

Searching for books is one of the most crucial functions of LMS. The user must be able to search for any book. Different users may want to search for a book through different methods. Therefore, the interviewer can ask questions like these:

- Can the user search for a book using attributes other than the book name?
- How can the user search for a book by its author name, publication date, etc.?
- How will the user search a specific category of books, like magazines, journals, newspapers, etc.?

2.2 Versatility

Before designing the system, it is mandatory to specify the actors of the system. Hence , the interviewer can ask about the actors of the system as follows:

- Can only a librarian or all library members use the software?

2.3 Book reservation

Another significant feature of LMS is the reservation of the book.

- What is the mechanism of book reservation?
- Can a member reserve a book again if it is already reserved?
- How does the status of the book change when a member returns a book?

2.4 Book renewal

Similar to the book reservation, the interviewer can ask about the book renewal functionality with a question like this:

- What is the mechanism of book renewal if a member wants to hold a book for longer?

2.5 Fine management

There is another question that the interviewer may be interested in asking:

- How is the calculation and deduction of the fine handled if the book is returned late?

3 Design approach

We will design this library management system using the bottom-up design approach. For this purpose, we will follow the steps below:

- First, we'll identify simple core entities such as Book, Member, and Librarian, and define their responsibilities.
- Next, we'll model book search, borrowing, return, reservation, renewal, and fine calculation workflows.
- We'll ensure the system supports role-based access control (RBAC) and maintains accurate transaction and catalog records.

This design approach will address concurrency and scalability and follow SOLID principles to ensure maintainability and extensibility. Later , diagrams and code will illustrate major workflows and class structures.

3.1 Design pattern

During an interview, discussing the design pattern a library management system falls under is always a good practice. Stating the design pattern gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking questions like, "Define design patterns."

Question:

Which design pattern(s) should be used to design a library management system? Please elaborate on your choice(s).

Let's explore the requirements of the library management system in the next lesson.

4 Requirements for the Library Management System

In this lesson, we will list the requirements of our library management system. This is a crucial step, as requirements define the scope of a problem. Getting them right from the interviewer and understanding them well will make the system design process smooth and easy.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

5 Requirement collection

For the LMS (library management system), the requirements have been defined below:

- **R1:** The system should store information about books and members, and maintain a complete log of all book borrowing, return, reservation, and renewal transactions.
- **R2:** Every book must have a unique identification number and detailed information, including rack/location in the library.
- **R3:** Each book should include the ISBN, title, author name, subject, and publication date.

Column component type 'MxGraphWidget' not yet supported.

- **R4:** A book can have multiple copies; each physical copy is a distinct book item with a unique ID.
- **R5:** The system must support two types of users: librarian and member, each with a library card and a unique card number.
- **R6:** Every user must have a library card with a unique card number.

Column component type 'MxGraphWidget' not yet supported.

- **R7:** Members can borrow a maximum of 10 books at a time.
- **R8:** A member can borrow a book for 15 days.
- **R9:** Only one member can reserve each book item at a time.
- **R10:** The system must record who issued, reserved, or renewed a book item and on which date.

Column component type 'MxGraphWidget' not yet supported.

- **R11:** According to policy limits, the system must allow members to renew borrowed books.
- **R12:** The system should notify members if a book is not returned by the due date or when a reserved book becomes available.
- **R13:** If a book is unavailable, a member should be able to reserve it for when it becomes available.
- **R14:** The system should allow users to search for books by title, author, subject, or publication date.

Column component type 'MxGraphWidget' not yet supported.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases of our library management system.

6 Use Case Diagram for the Library Management System

Let's build the use case diagram for the library management system and understand the relationships between its main actors, system functions, and user interactions. First , we will define the different elements of our library, followed by the complete use case diagram of the system.

6.1 System

Our system is the library management system. It manages the library's catalog, member accounts, book transactions, and notifications.

6.2 Actors

Next, we will define the main actors of our library management system.

Primary actors

- **Member:** A library member who can search, reserve, borrow, renew, or return books, pay fines, and request membership cancellation.

- **Librarian:** The administrator responsible for managing books and book items, issuing and returning books, handling reservations, managing member accounts, and overseeing fines.

Secondary actors

- **System:** It can send alerts related to reservations and late returns of books.

6.3 Use cases

In this section, we will define the library's use cases. We have listed them according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Member

- **Login/Logout:** To securely access or exit their member account and access library services.
- **Register/Update account:** To create a new membership account or update personal and contact information.
- **Cancel membership:** To voluntarily request termination of their library membership.
- **View account:** To review their own account details, borrowing history, current check-outs, reservations, and fines.
- **Search catalog:** Search the library catalog for books or resources by title, author, subject, or publication date.
- **Reserve book:** To place a hold on a currently checked-out book, ensuring it can be borrowed when it becomes available.
- **Checkout book:** To complete the process of borrowing a book, either directly or by following a successful reservation.
- **Renew book:** To request an extension for the borrowing period of a currently issued book.
- **Return book:** To return a borrowed book to the library, update the system, and potentially make it available for other members.
- **Remove reservation:** To cancel a previously placed hold on a book.
- **Pay fine:** To settle any outstanding fines incurred due to overdue book returns or other violations, allowing the member to maintain good standing.

Librarian

- **Login/Logout:** To securely access or exit the librarian account and manage system resources.
- **Register/Update account:** To register a new library member by collecting and saving their details in the system, or modify a member's personal details or membership status.
- **Cancel membership:** To terminate the library membership of a member and deactivate their account.
- **View account:** To access and review a library member's account details and history.
- **Issue library card:** To create and assign a unique one to a new member during registration, enabling them to borrow books and access other library services.
- **Add book:** To add a new book entry to the library's catalog, including essential metadata such as author, ISBN, etc.
- **Edit book:** To update the information of an existing book in the library catalog.
- **Remove book:** To delete a book record from the library catalog, typically if it is no longer part of the library's collection.
- **Add book item:** To add a physical copy of a book to the library's inventory (e.g., when purchasing additional copies).
- **Edit book item:** To update a specific physical book copy's details (such as rack location).
- **Remove book item:** To delete a specific book copy from the inventory.
- **Issue book:** To lend a book item to a library member, recording the issuance in the system.
- **Renew book:** To extend the borrowing period for a book already issued to a member, per library policy.
- **Update catalog:** To comprehensively manage the library catalog by adding, editing, or removing books or book items.
- **Remove reservation:** To cancel a member's book reservation, make the book available to others.

Library management system

- **Calculate fine:** To automatically determine the fine amount for overdue books based on the return date and the library's fine policy.
- **Send overdue notification:** To send alerts to members when the return due date for a borrowed book has passed, prompting action.
- **Send reservation available notification:** To inform members when a previously reserved book becomes available for borrowing or checkout.
- **Send reservation canceled notification:** To notify members when their reservation for a book has been canceled.

6.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

Search catalog is a generalized use case. Members can search by different attributes. Specialized use cases include:

- **Search by title**
- **Search by author**
- **Search by subject**
- **Search by publication date**

This means that "search catalog" generalizes the more specific search methods.

Associations

The table below shows the association relationship between actors and their use cases.

Librarian	**Member**	**Library Management System**
Log in/Log out	Log in/Log out	Calculate fine
Register/Update account	Register/Update account	Send overdue notification
Cancel membership	Cancel membership	Send reservation available notification
View account	View account	Send reservation canceled notification
Issue library card	Search catalog	Add book
Check out book	Reserve book	Edit book
Renew book	Add book item	Remove book
Return book	Edit book item	Remove reservation
Pay fine	Issue book	Renew book
Update catalog	Remove reservation	Remove book item

Include

When a librarian adds, edits, or removes a book from the catalog, they must also manage the associated book items (physical copies). These steps are best represented using "include" relationships in a use case diagram:

- **Add book, Edit book, and Remove book** use cases all include the **Add book item, Edit book item, and Remove book item** use cases, respectively.
- **Add book item**, **Edit book item**, and **Remove book item** each include **Update catalog**, ensuring the catalog stays consistent after each change.

When registering a new member, the librarian must issue a library card to enable borrowing privileges:

- The **Register new account** use case includes the **Issue library card** use case.

When issuing a book to a member, the librarian or system completes the checkout process:

- The **Issue book** use case includes the **Checkout book** use case.

During checkout, if the member had reserved the book, the system removes the reservation:

- The **Checkout book** use case includes the **Remove reservation** use case.

For overdue books, when a member returns a book late, the system automatically calculates the fine:

- The **Return book** use case includes the **Calculate fine** use case.

Extend

When a member returns a book, additional steps may be required if certain conditions are met. These conditional actions are best represented using **«extend»** relationships in a use case diagram:

- The **Pay fine** use case extends the **Return book** use case if the book is returned after the due date.

For the reservation process, if a reserved book becomes available, the system will notify the member:

- The **Send reservation available notification** use case extends the **Reserve book** use case when the reserved book becomes available.

When a member cancels a reservation, the system sends a notification:

- The **Send reservation canceled notification** use case extends the **Remove reservation** use case.

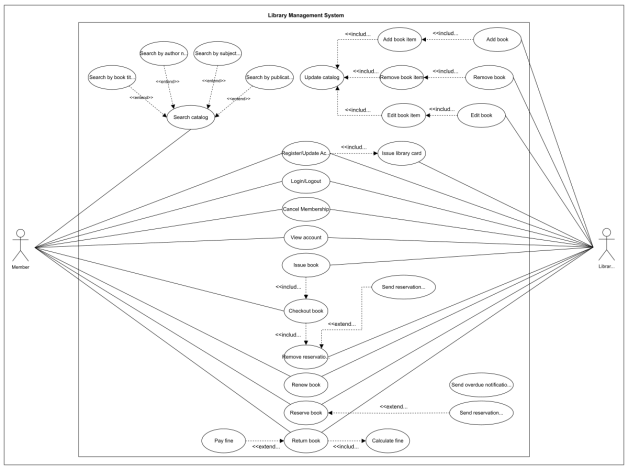


Figure 1.2: Use case diagram of the library management system

6.5 Use case diagram

Here is the use case diagram of the library management system:

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

7 Class Diagram for the Library Management System

Here, we'll create the class diagram for our system based on the previously gathered requirements. In the class diagram, we will first design/create the classes, abstract classes, and interfaces for the system, and then we'll identify the relationship between classes in accordance with all requirements of the library management system.

7.1 Components of a library management system

In this section, we will define the classes for LMS. Following the bottom-up approach for designing a class diagram, we'll first create the classes of small components. After that, we will integrate those components and create the class diagram for the library management system.

Book and book item

The `Book` class represents the conceptual essence of a book. This includes metadata or descriptive information about a book that isn't specific to any physical copy. It serves as a blueprint that captures the shared attributes of all instances or copies of the book, whether in the world or within the library's collection.

The `BookItem` class, in contrast, represents a specific physical or digital instance of a `Book`

in the library's collection. Rather than extending the `Book`, it has a composition relationship with it, meaning each `BookItem` is tightly associated with a `Book`. All corresponding `BookItem` instances must also be deleted if a `Book` is deleted. The `BookItem` class handles the properties and behaviors associated with individual copies that patrons can borrow, reserve, or reference within the library. Each `BookItem` has unique attributes to manage and track its status within the library system. The UML representation of `Book` and `BookItem` is shown in the class diagram below:

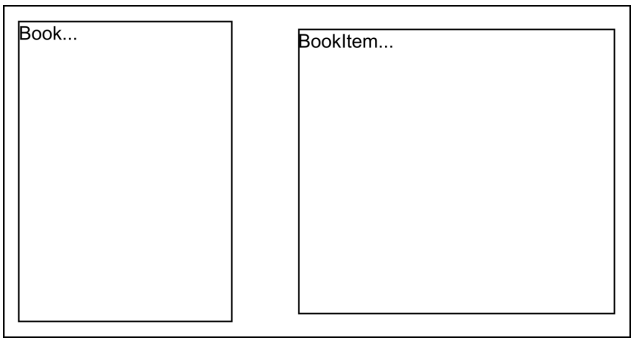


Figure 1.3: The class diagram of the Book and BookItem classes

R3: Every book should have an associated #key# ISBN: International Standard Book Number #key#, title, author name, subject, and publication date.
R4: Multiple copies of the book can be made, and each copy will be recognized as a book item.

Rack

We have seen a complex object, `Rack`, that was defined in the `BookItem` class. Now , we are going to create a `Rack` class. This class is used to identify the physical location of any book item in the library. Every rack has a specific rack number and a location identifier to represent the location of the book item in the library. The visual representation of the class is as follows:

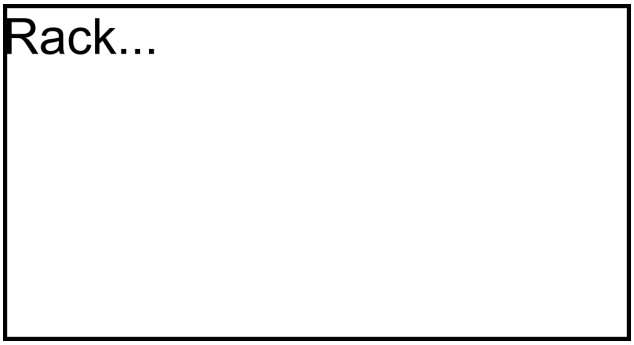


Figure 1.4: The class diagram of the Rack class

R2: Every book is supposed to have a unique identification number and other details, including a rack number, to help locate it physically.

Person and author

The `Person` class stores information related to a person, such as a name, email, phone number, etc. It also contains an object of the `Address` class to specify the person's address. There is also a class named `Author` that stores the author's data, such as the author's name and description. The author's information is also used in the `Book` class.
The representation of both classes is as follows:

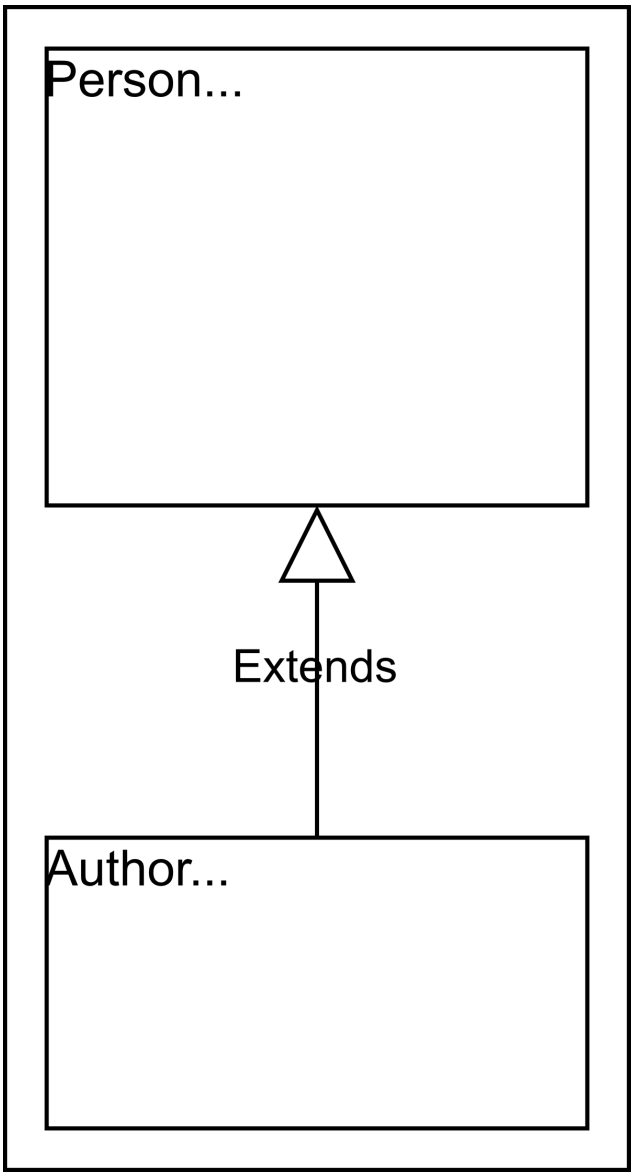


Figure 1.5: The class diagram of the Person and Author classes

User, librarian, and library member

`User` is an abstract class that represents the LMS system users. There are two types of users: librarians and library members.

The `Librarian` class is a derived class of the `User` class. It is responsible for adding new book items and blocking or unblocking any library member.
Similar to the `Librarian` class, the `Member` class extends the `User` class. The variable `totalBooksCheckedout` stores the number of books a member has already checked out. A member can reserve a book, return a book, or renew an already reserved book.
Since the `Librarian` and `Member` classes extend the `User` class, their class diagram representation would be as follows:

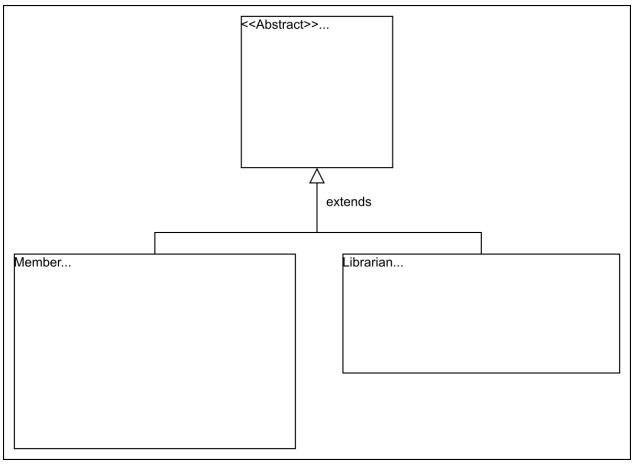


Figure 1.6: The class diagram of the User, Librarian, and Member classes

R5: There can be two types of users: the librarian and the members.
R11: The system should allow the user to renew the reserved book.

Library card

To manage each user's library card information, we have a `LibraryCard` class. Each library card has an identification number, issue date, and information on whether or not it is active. The class representation of the `LibraryCard` class is as follows:

R6: Every user must have a library card with a unique card number.

Book reservation

Book reservation is one of the library management system's most important requirements. To fulfill this functionality, we have a class named `BookReservation`. This class is responsible for managing the book reservation status of book items.
The UML representation of the class is shown below:

R10: The system should be able to record who issued or reserved a particular book and on which date.

Book lending

Similar to book reservations, book lending is also part of the system since the `BookLending` class manages the process of checking out book items. This class handles or processes infor-

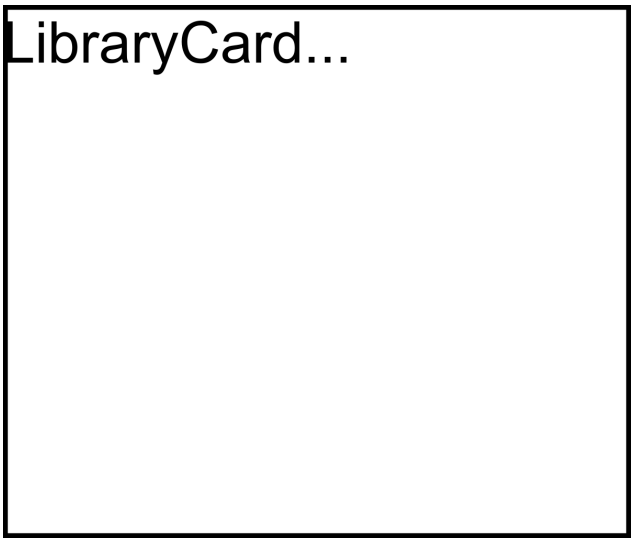


Figure 1.7: The class diagram of the LibraryCard class

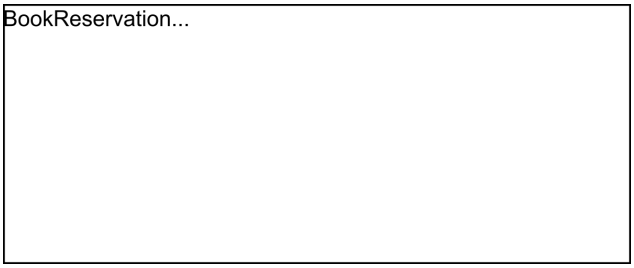


Figure 1.8: The class diagram of the BookReservation class

mation like the book lending date, due date, return date, etc.
Here is what the class definition looks like:

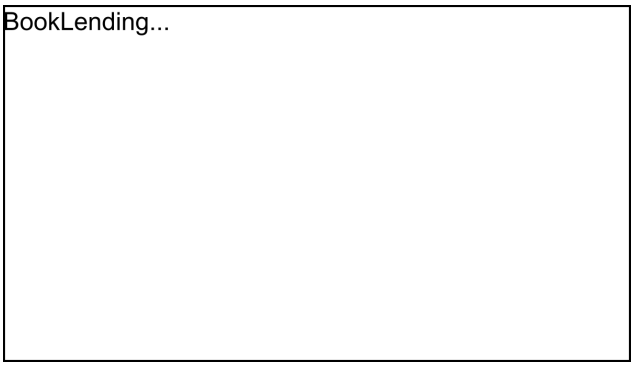


Figure 1.9: The class diagram of the BookLending class

Notification

`Notification` is an abstract class. If the book is not returned within the due date, the class notification is responsible for informing library members by sending a notification. Every notification has an ID, creation date, and content. The notification can be either a *postal notification* or an *email notification*.
The `PostalNotification` class requires the address of the library member to send a notification, while `EmailNotification` needs the email address of the library member to send a notification.
The relationship diagram of these classes is shown below:

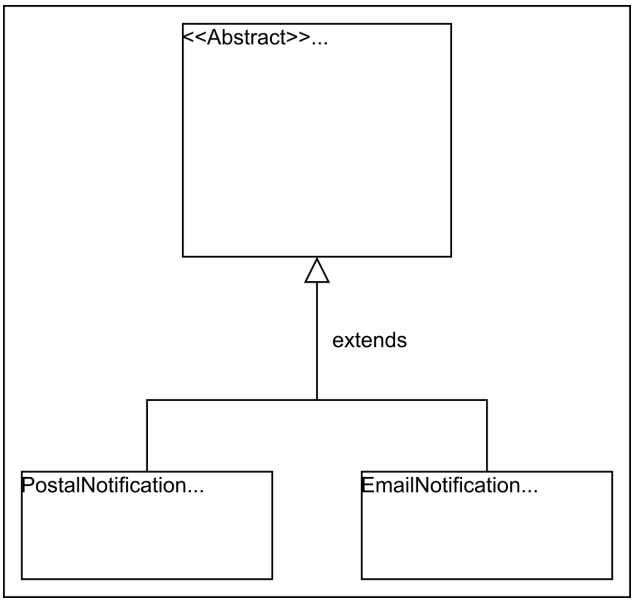


Figure 1.10: The class diagram of the Notification, PostalNotification, and EmailNotification classes

R12: The system should notify if the book is not returned within the due date.

Search and catalog

Search is one of the most important functionalities of the system. `Search` is the interface that allows the user to search for any book and return the list of books upon searching by any of the following methods:

- Search for a book by its title.
- Search for a book by its author's name.
- Search for a book by itssubject.
- Search for a book by its publication date.

`Catalog` is a class where the search functionality is implemented. In each catalog, the books are sorted according to one search technique, i.e., based on the book's title, author, subject, or publication date.
The following UML diagram shows this relationship:

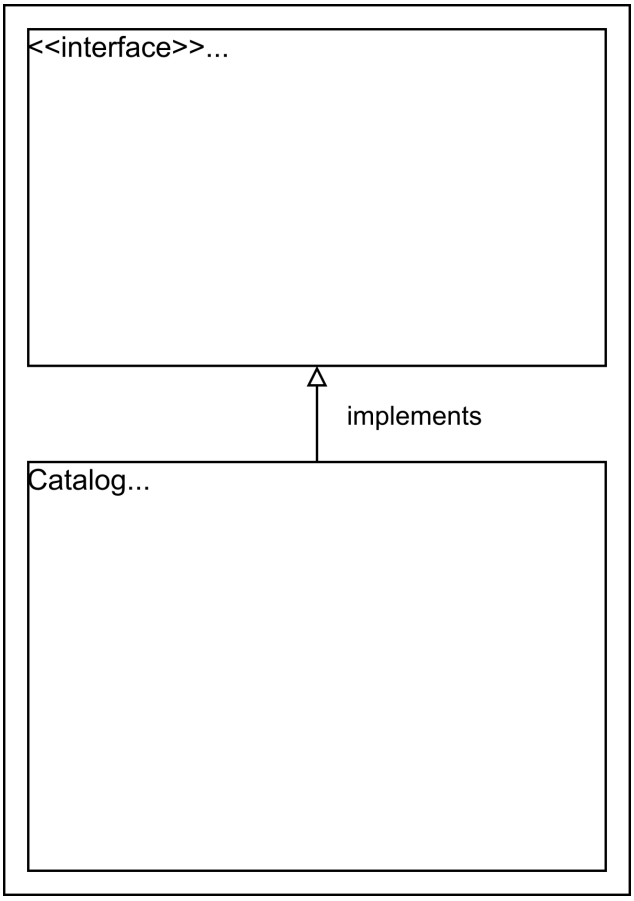


Figure 1.11: The class diagram of the Search and Catalog classes

R14: The system should allow users to search for a book by title, author name, subject, or publication date.

Library

The `Library` class is the base class of the system, which is used to represent the library. It is a central part of the organization. This class consists of two members: name and `Address`. The string type name stores the library's `name`, while the complex object `Address` stores the library's complete address location. The UML representation of the `Library` class is as follows:

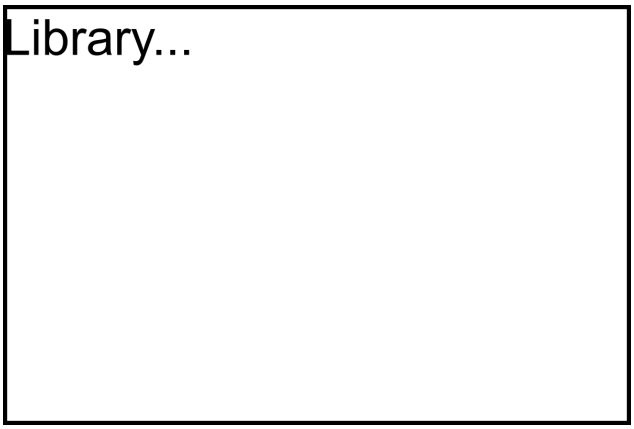


Figure 1.12: The class diagram of the Library class

Enumerations

Enumeration is generally a data type in which only a specific set of constants can be stored. The following is a list of enumerations required in LMS:
`BookFormat`: A book can only be of one of the specified formats. It can be a hardcover, paperback, audiobook, e-book, newspaper, magazine, or journal.
`BookStatus`: The book status describes the status of the particular book item for the user, whether it is available, reserved, loaned, or lost.
`ReservationStatus`: This tells about the reservation state of any book item, whether in a waiting state, pending state, canceled state, or none of them.
`AccountStatus`: The account status tells about the user account status, whether it is active, closed, canceled, blocklisted, or none.

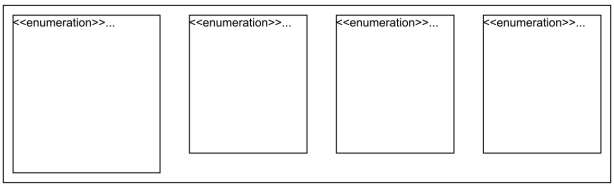


Figure 1.13: Enums in the library management system

Custom data type

The `Address` is a custom data type that stores the address of a library and its users.

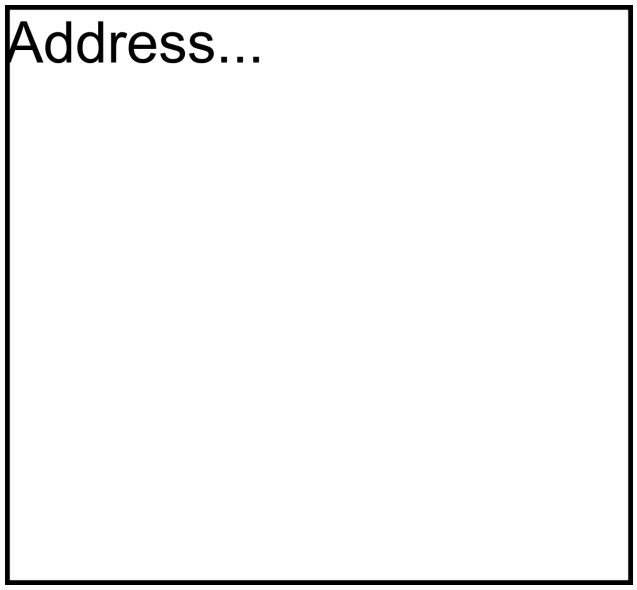


Figure 1.14: Class diagram of the Address custom data type

7.2 Relationship between the classes

Now, we will discuss the relationships between the classes we defined above in our library management system.

Association

The class diagram has the following association relationships:

One-way association

- The `User` has a one-way association with `BookItem` and `BookReservation`.
- Both `BookReservation` and `BookLending` have a one-way association with the `BookItem`.

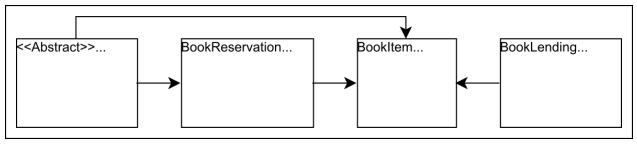


Figure 1.15: The one-way association relationship between the classes

Two-way association

- `Author` has a two-way association with `Book`.
- Both `Rack` and `Librarian` have a two-way association with `BookItem`.

- The `Notification` has a two-way association with `BookLending` and `BookReservation`.
- The `BookLending` has a two-way association with `BookReservation` and `User`.

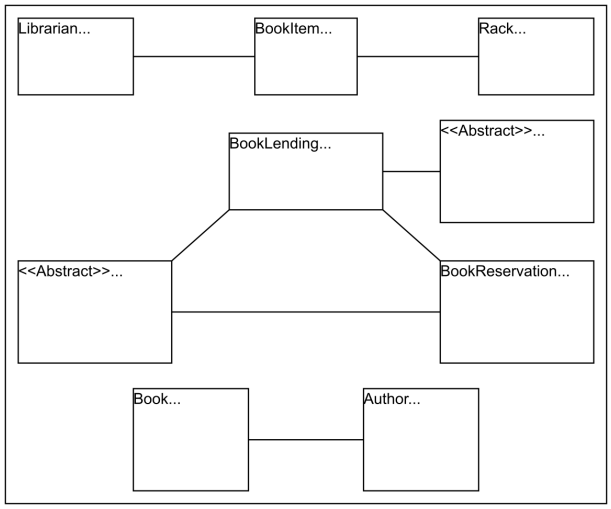


Figure 1.16: The two-way association relationship between the classes

Composition

- `Library` is composed of `BookItem`.
- `User` is composed of `LibraryCard`.
- `Book` is composed of `BookItem`.

Aggregation

- The `Catalog` class contains the `Book` class.

Inheritance

The following classes show an inheritance relationship:

- Both `Librarian` and `Member` classes extend the `User` class.
- Both `EmailNotification` and `PostalNotification` classes extend the `Notification` class.
- The `Catalog` class implements the `Search` interface.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

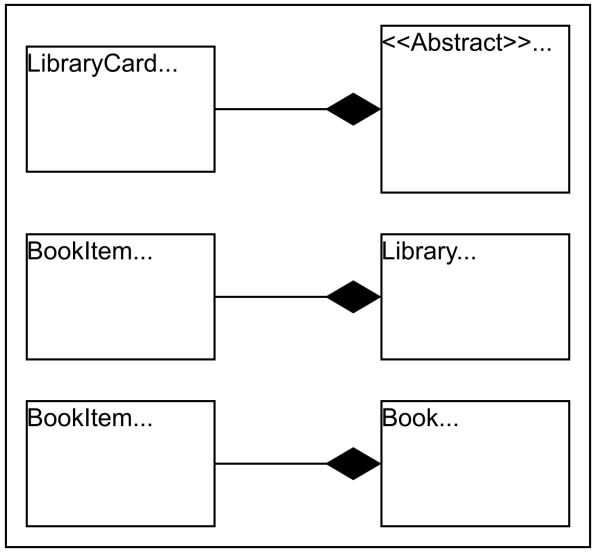


Figure 1.17: The composition relationship between the classes

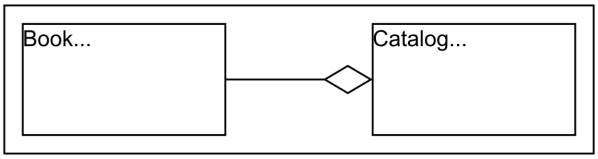


Figure 1.18: The aggregation relationship between the classes

7.3 Class diagram of the library management system

This section outlines the multiplicity (cardinality) relationships between the main classes in our Library Management system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Reason**
Library	BookItem	1 – 1..*	A library must have at least one book item, and can have many.
Rack	BookItem	1 – 0..*	Each rack may have zero or many book items.
Book	BookItem	1 – 1..*	Each book item is a copy of exactly one book.
Book	Author	1..* – 1..*	Every book has at least one author (possibly more). Similarly, every author can write one or many books.
User	LibraryCard	1 – 1	Each user has one library card.
User	BookLending	1 – 0..*	A user can have zero or more lending records.
BookReservation	BookItem	1 – 0..1	Each reservation is for one specific book item.
Catalog	Book	1 – 1..*	A catalog contains one or more books for searching.
Person	Author	1 – 1..*	Every author is a person (inheritance: one person can be multiple author roles if modeled).

Here is the complete class diagram for our library management system:

7.4 Design pattern

In our library management system, several well-known object-oriented design patterns are used to ensure the system remains extensible, maintainable, and robust:

- **Factory pattern:** We use the Factory pattern to create key domain objects such as `Book`, `BookItem`, `Member`, and `Librarian` in a consistent and controlled manner. For example, a `BookFactory` class can encapsulate the logic for creating books with all required metadata and validations. This approach centralizes creation logic, prevents inconsistent states, and supports future extensions such as special editions or new formats.

- **Delegation pattern:** The Delegation pattern helps distribute responsibilities across collaborating classes. For instance, while the `Librarian` class initiates actions like

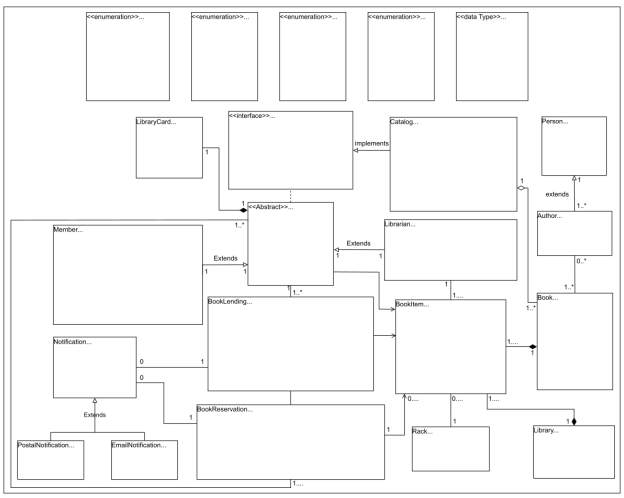


Figure 1.19: The class diagram of the library management system

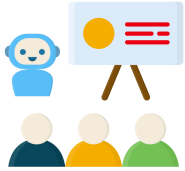
adding or removing book items, the actual details (such as updating inventory or status) are delegated to the `BookItem` class. This means `Librarian` orchestrates, but each `BookItem` manages its own data and behavior, promoting separation of concerns.

- **Observer pattern:** The Observer pattern is applied for notification workflows. Members who reserve or are interested in a specific book can be registered as observers. When the status of a `BookItem` changes (for example, a reserved or unavailable book becomes available), the system automatically notifies all relevant observers (e.g., via email or postal notification). This ensures real-time, event-driven communication and a responsive user experience.

These patterns work together to address system complexity, encourage reuse, and make it easier to maintain and extend the system in the future. Later sections and code examples will highlight where these patterns are implemented within the class structure.

7.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



7.6 Additional requirements

The interviewer can introduce some additional requirements in the LMS, or they can ask some follow-up questions. Let 's see an example of additional requirements:

BarcodeReader: Each member should have a unique barcode on their library card, and each book should also have a distinct barcode associated with it; the system should be able to scan the barcode of every book and member. To fulfill this requirement, we have the class diagram shown below:

Column component type 'MxGraphWidget' not yet supported.

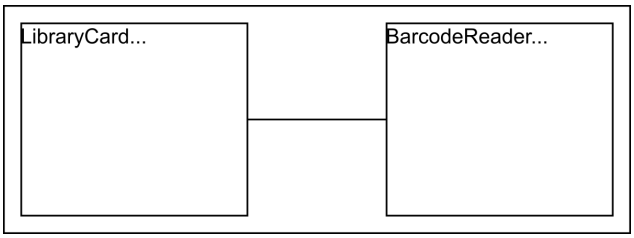


Figure 1.20: Relationship between the BarcodeReader and the LibraryCard class

FineTransaction: As there is a fine for not returning the book within the time limit, some mechanism should exist to pay the fine. There are three ways to pay the fine: check, cash, or credit card.

Column component type 'MxGraphWidget' not yet supported.

The fine payment functionality follows the **Decorator pattern** as the fine keeps adding upon the increased number of days. The class diagram provided below shows the relationship of `FineTransaction` with the `Fine` class:

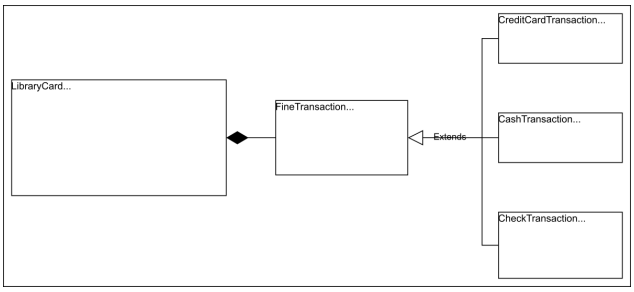


Figure 1.21: Relationship of the FineTransaction class with other classes

Quiz: Question 1: Let's say that the interviewer asks you what would happen if two or more members try to reserve the same book item. Which member will get the book? **Answer:** If two or more members try to reserve the book item at the same time,

the process will be executed on a first come, first serve (FCFS) basis.

For example, the member who comes first gets the book reserved. If two members come simultaneously, the system will check which member has the fewest checked-out books through 'totalBooksCheckedOut' and reserve the book item for that particular member. If both members have already reached the maximum number of checkouts, no one can reserve a book.

Note: The member can reserve a book that is currently unavailable, so they will get it first whenever it becomes available.

We have successfully designed the class diagram for the library management system. Let 's move to the next lesson to see how we can construct the sequence diagram for the system.

8 Sequence Diagram for the Library Management System

Sequence diagrams help visualize the flow of interactions between different entities and objects in the system, step by step. This lesson will illustrate the main interactions in lending and returning a book within the library management system. Understanding these workflows is essential for modeling object responsibilities and collaborations.
For this lesson, we will focus on the following two scenarios:

- **Issuing (Lending) a book:** The process through which a member requests to borrow a book from the library.
- **Returning a book:** The process for when a member returns a borrowed book to the library.

8.1 Issuing (Lending) a book

The sequence diagram for issuing a book involves the following participants:

- **Actors:** `Member` and `Librarian`
- **Objects:** `Book` and `BookItem`

The typical workflow is as follows:

1. **Member requests to issue a book:**
   (a) The member initiates the process by asking the librarian to issue a specific book.
2. **Librarian checks the member's quota:**

(a) The librarian verifies whether the member has reached their maximum allowed number of borrowed books.

i. If the quota is reached:

A. The librarian informs the member that no more books can be issued now.

ii. If quota is available:

A. The librarian proceeds to check the status of the requested book.

3. **The librarian checks the book's status:**

(a) If the book is *available*:

i. The librarian issues the book to the member, updating lending records accordingly.

(b) If the book is *reserved* (by another member):

i. The librarian informs the requesting member that the book cannot be issued.

The sequence diagram below visualizes these interactions, clearly showing the flow of requests, validations, and responses between the member, librarian, and book entities.

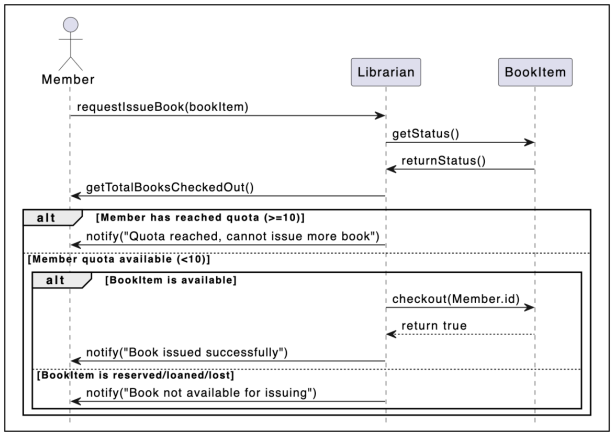


Figure 1.22: The sequence diagram for issuing a book

8.2 Sequence challenge: Return a book

You will complete a sequence diagram for returning a book to the library. The diagram below demonstrates a skeleton for this process.

Notice that the arrows in the above diagram are numbered from 1 to 13. Below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct order sequence they should appear in the sequence diagram's skeleton above?

Note: If you're stuck, click the "Show Solution" button.

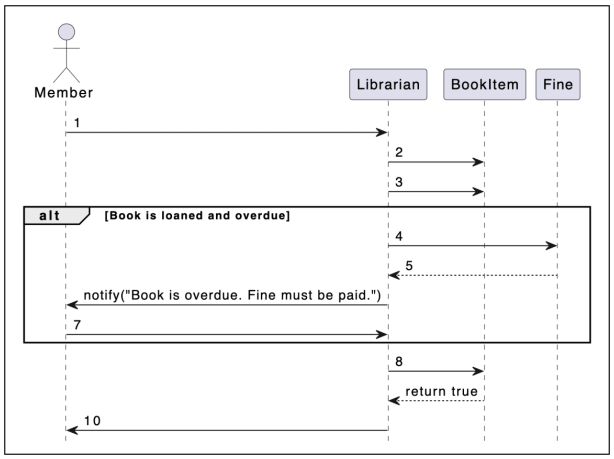


Figure 1.23: The sequence diagram for returning a book

Alternatively, you can also click the "Show complete diagram" button below to view the complete sequence diagram of the return book sequence.

We will create activity diagrams for our library management system in the next lesson.

9 Activity Diagram for the Library Management System

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our LMS(Library Management System). In this lesson, we will create activity diagrams for the following three activities:

- Check out a book from the library.
- Return a book to the library.
- **Activity challenge:** Renew a book from the library.

9.1 Check out a book from the library

The following are the states and actions involved in this activity diagram.

States

Initial state: The member selects a book and initiates checkout.
Final state: There are two final states present in this activity diagram, shown below:

- The member completes the checkout process successfully, and the book will be allocated to the member.
- An error occurred during checkout due to book unavailability, or the book limit was exceeded.

Actions

The member selects a book and enters the ID. The system performs a few checks, such as book availability, the member's maximum limit, and book reservations. If all checks are clear, the book will be issued. Otherwise, the system will show an error message.
The activity diagram of a library book checkout is given below based on the order shown above.

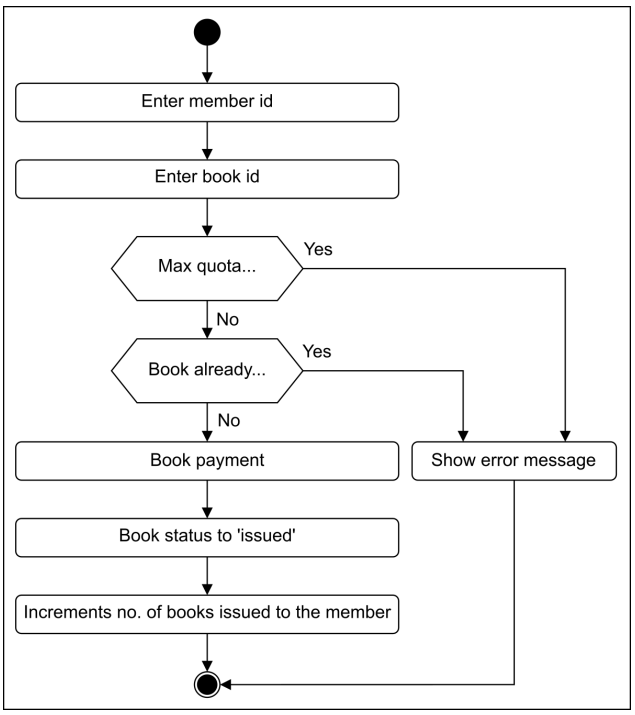


Figure 1.24: The activity diagram to check out a book from the library

9.2 Return a book to the library

The following are the states and actions involved in this activity diagram.

States

Initial state: The member returns a book to the library.
Final state: There are two final states present in this activity diagram, shown below:

- The member completes the return process and pays a fine, if any.
- The system allocates a book to someone who reserved that book.

Actions

The member enters the book ID. The system checks if the book is returned within the due date, and the member pays a fine, if any. Then , the book is allocated to someone who has reserved it.
Based on the order above, the activity diagram below demonstrates returning a book to the library.

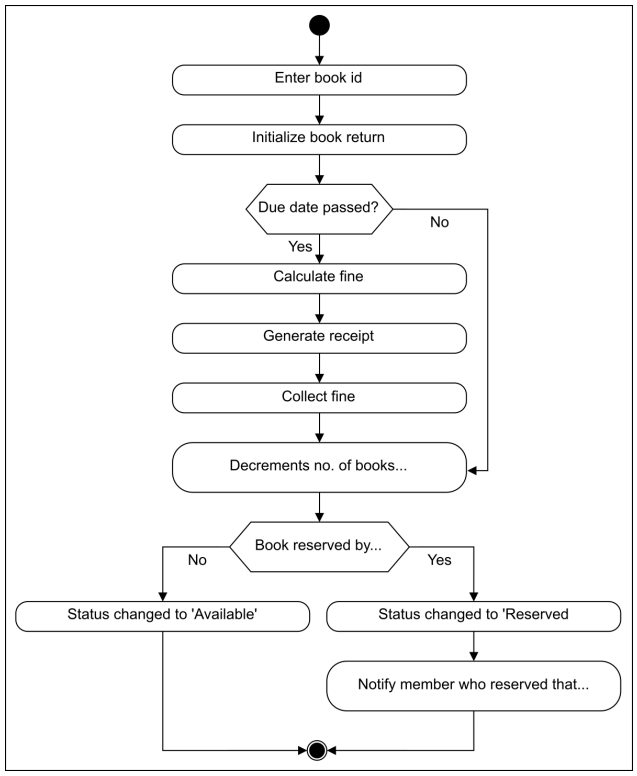


Figure 1.25: The activity diagram for returning a book to the library

9.3 Activity challenge: Renew a book from the library

You will create an activity diagram of a member renewing a book from the library.
The skeleton of the activity diagram given below demonstrates a customer looking for book renewal from the library.

Notice that the actions in the above diagram are numbered from 1 to 7. The slots below represent the activities, and the arrows represent the flow from one activity to another. Can you rearrange the slots provided below in the correct order they should appear in the activity diagram above?
Based on the description above, can you fill in the missing slots in the activity diagram with the correct sequence of actions?

Note: If you're stuck, click the "Show Solution" button.

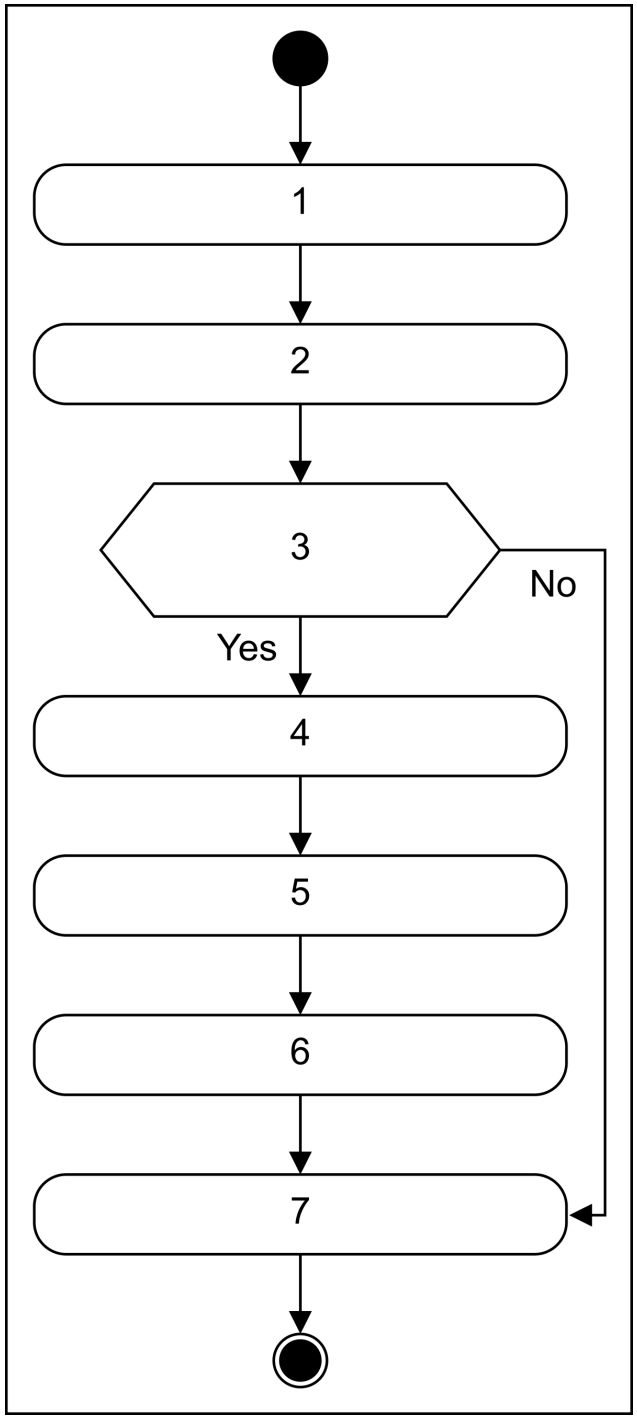


Figure 1.26: The activity diagram for renewing a book at the library

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of our library management system's activity diagrams. The next lesson will present the code for our designed classes in some of the most popular languages.

10 Code of Library Management System

We've gone over the different aspects of the library management system and observed the attributes attached to the problem using various UML diagrams. Let us now explore the more practical side of things, where we will work on implementing the library management system using multiple languages. This is usually the last step in an object-oriented design interview process.

We have chosen the following languages to write the skeleton code of the different classes present in the library management system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click **Executable Code: Library Management System**.

10.1 Library management

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations

First, we will define all the enumerations required in the library management system. According to the class diagram, four enumerations are used in the system: `BookFormat`, `BookStatus`, `ReservationStatus`, and `AccountStatus`. The code to implement these enumerations is as follows:‘

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Enum definitions

```
// definition of enumerations used in library management system
   enum BookFormat { HARDCOVER,
PAPERBACK, AUDIOBOOK,
EBOOK, NEWSPAPER,
MAGAZINE, JOURNAL
}

enum BookStatus { AVAILABLE,
RESERVED, LOANED,
LOST }

enum ReservationStatus { WAITING,
PENDING, CANCELED,
NONE }

enum AccountStatus { ACTIVE,
CLOSED, CANCELED,
BLOCKLISTED, NONE
}
```

Address and person

This section contains the code for `Address` and `Person` classes, where the `Person` class is composed of an `Address` class. The implementation of these classes can be found below:

The Address and Person classes

```
public class Address { private String streetAddress;
private String city; private String state;
private int zipCode; private String country;
}

public class Person { private String name;
private Address address; private String email;
private String phone; }
```

User

The `User` is an abstract class that represents the various people or actors that can interact with the system. As there are two types of users, the librarian and the library member, the user can either be a `Librarian` or a `Member`. The implementation of the mentioned classes is shown below:

User and its child classes

```
// User is an abstract class public abstract class User { private String id;
private String password; private AccountStatus status;
private Person person; private LibraryCard card;

public abstract boolean resetPassword(); }

public class Librarian extends User { public boolean addBookItem(BookItem bookItem);
public boolean blockMember(Member member); public boolean unBlockMember(Member member);
public boolean resetPassword() { // definition
 }
public boolean addBookItem(BookItem bookItem) { // definition
 }
}

public class Member extends User { private Date dateOfMembership;
 private int totalBooksCheckedOut;

public boolean reserveBookItem(BookItem bookItem); private void incrementTotalBooksCheckedOut();
public boolean checkoutBookItem(BookItem bookItem); private void checkForFine(String bookItemId);
public void returnBookItem(BookItem bookItem); public boolean renewBookItem(BookItem bookItem);
public boolean resetPassword() { // definition
 }
}
```

Book reservation, book lending, and fine

This component shows the implementation of `BookReservation`, `BookLending`, and Fine classes. These classes will be responsible for managing reservations against books, managing reservations, and calculating fines on books. The code is shown below:

The BookReservation, BookLending, and Fine classes

```
public class BookReservation { private String itemId;
private Date creationDate; private ReservationStatus status;
 private String memberId;

public static BookReservation fetchReservationDetails(String bookItemId); }

public class BookLending { private String itemId;
private Date creationDate; private Date dueDate;
private Date returnDate; private String memberId;
```

```
private BookReservation bookReservation; private User user;

public static boolean lendBook(String bookItemId, String memberId)
    ; public static BookLending fetchLendingDetails(String
    bookItemId);
}

public class Fine { private Date creationDate;
private String bookItemId; private String memberId;

public static void collectFine(String memberId, long days); }
```

Book and rack

The `Book` is an abstract class, and `BookItem` represents each book copy. For example, if there are two copies of the same book, there would be only one `Book` object and two `BookItem` objects. The code to implement these classes is as follows:

The Book, BookItem, and Rack classes

```
// User is an abstract class
public abstract class Book {
    private String isbn;
    private String title;
    private String subject;
    private String publisher;
    private String language;
    private int numberOfPages;
    private BookFormat bookFormat;
    private List<Author> authors;
}

public class BookItem {
    private String id;
    private boolean isReferenceOnly;
    private Date borrowed;
    private Date dueDate;
    private double price;
    private BookStatus status;
    private Date dateOfPurchase;
    private Date publicationDate;
    private Rack placedAt;
    private Book book;  // Agggregation: BookItem has a reference
        to a Book

    // Constructors, getters, and other existing methods...

    public boolean checkout(String memberId) {
```

```
        // Implementation for checkout logic
        // Update the status, set due date, etc.
        // Return true if checkout is successful, false otherwise
        return true;  // Placeholder, replace with actual logic
    }

    public void setPlacedAt(Rack rack) {
        this.placedAt = rack;
        // Additional logic if needed
    }

    public void setAddedBy(Librarian librarian) {
        // Implementation for setting the librarian who added the
            book item
        // This might involve updating logs or other data related
            to the librarian
        // No return value as it's a setter method
    }

    // Other methods...

    public Book getBook() {
        return book;
    }
}

public class Rack {
    private int number;
    private String locationIdentifier;
    private List<BookItem> bookItems;
    public void addBookItem(BookItem bookItem) {
        // definition
    }
}
```

Notification

The `Notification` class is another abstract class responsible for sending notifications to the users, with the `PostalNotification` and `EmailNotification` classes as its child classes. The implementation of this class can be found below:

Notification and its derived classes

```
// User is an abstract class public abstract class Notification {
    Private String notificationId;
Private Date creationDate; Private String content;
private BookLending bookLending; private BookReservation
    bookReservation;
```

```

public abstract boolean sendNotification(); }

public class PostalNotification extends Notification { private Address address;

public boolean sendNotification(){ // definition
 }
}

public class EmailNotification extends Notification { private String email;

public boolean sendNotification(){ // definition
 }
}
```

Search and catalog

The `Search` is an interface used to efficiently search library books by various methods, and the `Catalog` class is used to implement the search interface to help in book searching. The code to perform this functionality is presented below:

The Search interface and the Catalog class

```
public interface Search { // Interface method (does not have a body)
public List<Book> searchByTitle(String title); public List<Book> searchByAuthor(String author);
public List<Book> searchBySubject(String subject); public List<Book> searchByPublicationDate(Date publishDate);
}

public class Catalog implements Search { private HashMap<String, List<Book>> bookTitles;
private HashMap<String, List<Book>> bookAuthors; private HashMap<String, List<Book>> bookSubjects;
 private HashMap<String, List<Book>> bookPublicationDates;

public List<Book> searchByTitle(String query) { // definition
 }
public List<Book> searchByAuthor(String query) { // definition
 }
public List<Book> searchBySubject(String query) { // definition
 }
public List<Book> searchByPublicationDate(String query) { // definition
 }
```

```
}
```

Library

The final class of LMS is the `Library` class, which will be a Singleton class, meaning the entire system will have only one instance of this class. The implementation of this class can be found below:

The Library class

```
public class Library {
private String name; private Address address;

// Private constructor to prevent external instantiation private
   Library() {}

 public Address getAddress();

// The Library is a singleton class that ensures it will have only
    one active instance at a time private static Library library =
    null;

// Created a static method to access the singleton instance of
   Library class public static Library getInstance () {
if (library == null) {
library = new Library (); }
return library; }
}
```

10.2 Executable code: Library management system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the library management system. The system's main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System initialization:** Sets up books, copies, members, a librarian, and the catalog.
- **Scenario 1 (Member searches and reserves a book):** The member searches for a book, reserves it, and checks it out.
- **Scenario 2 (Member returns the book (on time and late)):** The member returns the book, and the system calculates if a fine is due.
- **Scenario 3 (Book renewal):** The member renews a borrowed book.

- **Notifications:** Overdue and reservation notifications are simulated.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable code: Library management system

```java
import java.util.*;
import java.text.SimpleDateFormat;

public class Driver {
    public static void main(String[] args) {
        try {
            // ==========================================
            // >>> SYSTEM INITIALIZATION
            // ==========================================
            System.out.println("\n========== SYSTEM INITIALIZATION ==========\n");
            Address libAddress = new Address("1 Main St", "Springfield", "State", 12345, "Country");
            Library library = Library.getInstance("Springfield Public Library", libAddress);

            // ---> Authors
            Author author1 = new Author("Jane Austen", libAddress, "austen@email.com", "123456", "Famous novelist");
            Author author2 = new Author("Mark Twain", libAddress, "twain@email.com", "234567", "Another novelist");

            // ---> Books and BookItems
            SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
            Book book1 = new Book("ISBN123", "Pride and Prejudice", "Novel", "Publisher A",
                    sdf.parse("2000-01-01"), "English", 300, BookFormat.HARDCOVER, Arrays.asList(author1));
            Book book2 = new Book("ISBN456", "Adventures of Huckleberry Finn", "Adventure", "Publisher B",
                    sdf.parse("2005-05-20"), "English", 250, BookFormat.PAPERBACK, Arrays.asList(author2));
            Rack rack1 = new Rack(1, "A1");
            Rack rack2 = new Rack(2, "B2");
            BookItem bookItem1 = new BookItem("BI001", book1, rack1, 30.0, sdf.parse("2020-01-01"), book1.publicationDate);
            BookItem bookItem2 = new BookItem("BI002", book2,
```

```
                rack2, 25.0, sdf.parse("2021-06-15"), book2.publicationDate);
            library.catalog.addBookItem(bookItem1);
            library.catalog.addBookItem(bookItem2);

            // ---> Users
            LibraryCard cardMember = new LibraryCard("CARD1001", new Date());
            Person personMember = new Person("Alice", libAddress, "alice@email.com", "345678");
            Member member = new Member("MEM001", "pass", personMember, cardMember);

            LibraryCard cardLibrarian = new LibraryCard("CARD2001", new Date());
            Person personLibrarian = new Person("Libby", libAddress, "libby@email.com", "456789");
            Librarian librarian = new Librarian("LIB001", "pass", personLibrarian, cardLibrarian);

            // ============================================
            // >>> SCENARIO 1: Search and Reserve
            // ============================================
            System.out.println("\n------------------------------");
            System.out.println(">>> SCENARIO 1: Member searches and reserves a book");
            System.out.println("------------------------------\n");

            List<BookItem> foundBooks = library.catalog.searchByTitle("Pride and Prejudice");
            if (!foundBooks.isEmpty()) {
                BookItem bookItem = foundBooks.get(0);

                System.out.println("-> Member [" + member.person.name + "] attempts to reserve book item [" + bookItem.id + "]");
                member.reserveBookItem(bookItem);

                System.out.println("-> Member [" + member.person.name + "] checks out reserved book item [" + bookItem.id + "]");
                member.checkoutBookItem(bookItem);

            } else {
                System.out.println("Book not found.");
            }
```

```java

            // ==========================================
            // >>> SCENARIO 2: Return Book (Late Return)
            // ==========================================
            System.out.println("\n------------------------------");
            System.out.println(">>> SCENARIO 2: Member returns the book late");
            System.out.println("------------------------------\n");

            // Fast forward due date to simulate late return
            Calendar lateCalendar = Calendar.getInstance();
            lateCalendar.setTime(new Date());
            lateCalendar.add(Calendar.DATE, -3); // Simulate due date as 3 days ago
            bookItem1.dueDate = lateCalendar.getTime();

            System.out.println("-> Member [" + member.person.name + "] returns book item [" + bookItem1.id + "] after due date.");
            member.returnBookItem(bookItem1);

            // ==========================================
            // >>> SCENARIO 3: Renew Book
            // ==========================================
            System.out.println("\n------------------------------");
            System.out.println(">>> SCENARIO 3: Member renews a book");
            System.out.println("------------------------------\n");

            // Check out again for demonstration
            System.out.println("-> Member [" + member.person.name + "] checks out the book again.");
            member.checkoutBookItem(bookItem1);

            System.out.println("-> Member [" + member.person.name + "] renews the book.");
            member.renewBookItem(bookItem1);

            // ==========================================
            // >>> NOTIFICATIONS
            // ==========================================
            System.out.println("\n------------------------------");
            System.out.println(">>> NOTIFICATIONS");
```

```
            System.out.println("-------------------------------\n");

            EmailNotification emailNotification = new EmailNotification(
                "N001", "Your book is overdue!", member.person.email);
            emailNotification.sendNotification();

            PostalNotification postalNotification = new PostalNotification(
                "N002", "Please return your book!", member.person.address);
            postalNotification.sendNotification();

            System.out.println("\n========== END OF DEMO ==========\n");

        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

10.3 Wrapping up

In this chapter, we've explored the complete design of a library management system. We 've also looked at how a basic library management system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

11 Summary: Designing a Library Management System

Now that you've completed the library management system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

11.1 Key requirements

This section outlines the primary functional requirements that define the scope and capabilities of the library management system.

1. Store and log all information about books, members, and transactions (borrowing, returns, reservations, renewals).
2. Ensure every book has a unique ID and detailed information, including its physical location.
3. Include each book's ISBN, title, author, subject, and publication date.
4. Support each book title's multiple physical copies (book items).
5. Define two user roles: librarian and member, each with a unique library card.
6. Require every user to have a library card with a unique number.
7. Limit members to borrowing a maximum of 10 books at one time.
8. Set the maximum borrowing period for a book to 15 days.
9. Allow a book item to be reserved by only one member at a time.
10. Record which user issued, reserved, or renewed a book item and the transaction date.
11. Enable members to renew borrowed books based on the library policy.
12. Send notifications for overdue books and available reservations.
13. Allow members to reserve a book if it is currently unavailable.
14. Enable users to search for books by title, author, subject, or publication date.

11.2 System actors

This section identifies the primary and secondary actors who interact with the system.

- **Member:** A library user who can search the catalog, borrow, return, renew, and reserve books. They can also manage their account and pay fines.
- **Librarian:** An administrator responsible for managing the library's collection, including adding and removing books and book items. They also manage member accounts, issue books, and handle fines.

11.3 Important classes and relationships

This section details the core classes that form the system's backbone, describing their purpose and how they relate to one another.

Class	Description	Important Relationships
User	An abstract class representing a person interacting with the system.	Parent to Librarian and Member.
Librarian	A user with administrative privileges to manage books and members.	Inherits from User. Has an association with BookItem.
Member	A user who can borrow, reserve, and return books.	Inherits from User.
Book	Represents the conceptual information of a book (title, author, ISBN).	Has an association with Author. Aggregated in Catalog.
BookItem	Represents a specific physical copy of a book that can be loaned out.	Composition with Book. Has an association with Rack.
BookLending	Manages the checkout process, including due dates and return dates.	Has an association with User and BookItem.
BookReservation	Manages reservations for unavailable book items.	Has an association with User and BookItem.
Catalog	Implements the search functionality, allowing users to find books.	Implements the Search interface. Aggregates Book.
Notification	An abstract class for sending notifications to members.	Parent to EmailNotification and PostalNotification.
LibraryCard	Contains the user's library card information.	Composition with User.

11.4 Design highlights

This section covers the overall design strategy, the core software design principles, and the key design patterns used to structure the system.

Design approach

The system was designed using a bottom-up approach. This involved identifying core entities like `Book`, `Librarian`, and `Member`, modeling their workflows, and finally integrating these components into a complete, scalable system that adheres to SOLID principles.

Design principles

The design of the library management system incorporates the SOLID principles to ensure it is robust, maintainable, and scalable.

- **Single Responsibility principle (SRP):** Each class has a distinct responsibility. For example, the `BookLending` class handles only the logic for borrowing books, while the `BookReservation` class manages only reservations.

- **Open/Closed principle (OCP):** The system is open for extension but closed for modification. For instance, new types of notifications (like SMS) can be added by creating a new subclass of `Notification` without altering existing code.

- **Liskov Substitution principle (LSP):** Subtypes can be substituted for their base types. A `Librarian` or `Member` object can be used wherever a `User` object is expected, as both inherit from `User`.

- **Interface Segregation principle (ISP):** The `Search` interface is a good example, providing a specific contract for search functionality that the `Catalog` class implements, without forcing it to depend on unrelated methods.

- **Dependency Inversion principle (DIP):** The system depends on abstractions, not concretions. The `Catalog` class depends on the `Search` interface, allowing different search implementations to be used interchangeably.

Design patterns

The system leverages several design patterns to solve common problems and improve the overall architecture.

- **Factory pattern:** Used to create domain objects like `Book` and `Member`, encapsulating the creation logic and ensuring objects are instantiated correctly.

- **Delegation pattern:** Responsibilities are distributed among classes. A `Librarian` delegates the task of managing a book's status to the `BookItem` class.

- **Observer pattern:** This pattern is used for notifications. Members can "observe" a `BookItem`; when its status changes (e.g., becomes available), they are automatically notified.

11.5 Object interactions

This section describes the system's dynamic behavior by explaining how key objects collaborate to perform tasks, based on the sequence diagram for issuing a book.

- **Book issuance request:** A `Member` initiates the process by requesting to borrow a `Book` from a `Librarian`.

- **Book status check:** If the member's quota is not exceeded, the `Librarian` checks the status of the `BookItem`.

- **Completing the checkout:** If the `BookItem` is available, the `Librarian` issues the book to the `Member`. If the book is already reserved, the request is canceled.

11.6 System workflow

This section outlines the primary end-to-end scenarios in the system, reflecting the logic from the activity diagrams.

- **Checking out a book:** A member starts by providing their member ID and the book ID. The system verifies that the member's borrowing quota is not exceeded and that the book is not already reserved. If both checks pass, the book's status is updated to "issued," and the member's count of borrowed books is incremented.
- **Returning a book:** A member provides the book ID to initiate a return. The system checks if the due date has passed. If it has, a fine is calculated and collected. The member's count of borrowed books is then decremented. Finally, the system checks if another member has reserved the book. If so, its status is changed to "Reserved" and the reserving member is notified; otherwise, it is set to "Available."

12 Key Insights and Tips: Designing a Library Management System

You have completed the case study for designing a Library Management system. Now , we will distill the key takeaways from this exercise. This lesson will highlight common design mistakes to watch out for, provide specific interview tips to help you shine, test your knowledge with a short quiz, and point you to related case studies to solidify your understanding of the core OOD principles involved.

12.1 Common mistakes

Avoiding these common mistakes will help you develop a more efficient and scalable library management system:

- **Confusing a book with a book item:** Treating the abstract concept of a book and its physical copy as the same thing. You should create a `Book` class for shared metadata like title and ISBN, and a separate `BookItem` class for each physical copy that can be tracked and loaned.
- **Designing an inflexible search functionality:** Implementing only a single way to search for books, such as by title. You should use a `Search` interface that a `Catalog` class can implement to support searching by attributes like author, subject, and publication date.
- **Hardcoding user types instead of using inheritance:** Creating separate and unrelated classes for librarians and members. A better approach is to use an abstract `User` class and have `Librarian` and `Member` classes extend it, which promotes code reuse and a cleaner design.

- **Ignoring different notification methods:** Assuming all notifications will be sent the same way, for example, by email. You should use an abstract `Notification` class with concrete implementations like `PostalNotification` and `EmailNotification` to make the system more flexible.

- **Failing to enforce borrowing limits:** Allowing a member to check out an unlimited number of books. The system must enforce business rules, such as limiting a member to a maximum of 10 books at a time.

12.2 Interview tips

Here are some essential tips to help you prepare for an interview where you're asked to design a library management system:

Tip	Explanation
Clarify the actors and their permissions early on.	Before diving into code, explicitly define the system's actors, such as the Member and Librarian, and list their use cases. This demonstrates a structured approach to requirement gathering and ensures your design addresses all user interactions mentioned in the problem.
Discuss the use of enumerations for fixed sets of constants.	Mention that you would use enums for types like BookStatus (available, reserved, loaned) and AccountStatus (active, canceled, blocklisted). This shows attention to detail, improves code readability, and ensures type safety compared to simple strings or integers.
Propose relevant design patterns for the system.	Show your OOD expertise by suggesting patterns like the Observer pattern to notify members when a reserved book becomes available, or the Factory pattern to create different Book objects in a controlled manner.
Articulate the fine calculation and payment mechanism.	Explain how the "Pay the fine" use case extends the "Return book" use case, meaning it's an optional flow that occurs only if the book is overdue. This demonstrates your understanding of use case relationships and how to model conditional workflows.
Address how to handle additional requirements.	If the interviewer adds a new requirement, like a barcode scanner, explain how your design can accommodate it. You could suggest adding a barcode attribute to the BookItem and LibraryCard classes and a new BarcodeReader class to process them, showing your adaptable design.

12.3 Test your understanding

Take a short quiz to evaluate your understanding of the library management system design.

Designing a Library Management System

Question 1: In the proposed class diagram, what is the most accurate relationship between the `Book` class (which holds metadata like ISBN and title) and the `BookItem` class (which represents a specific physical copy)?

- Inheritance
- Aggregation
- **[CORRECT]** Composition
  Explanation: A `BookItem` is a physical instance of a `Book` and its lifecycle is tied to the `Book`. It cannot exist independently, which defines a composition relationship.
- Association

Question 2: According to the use case diagram, which action can be performed by a `Member` but *not* by a `Librarian`?

- Renew book
- Reserve book
- **[CORRECT]** Return book
  Explanation: The use case diagram shows that the "Return book" use case is exclusively associated with the `Member` actor.
- Cancel membership

Question 3: Which design pattern is suggested to ensure that `Book` objects are created in a uniform and controlled manner?

- Observer
- **[CORRECT]** Factory
  Explanation: The document suggests using a `BookFactory` class based on the Factory design pattern to create `Book` objects consistently.
- Delegation
- Decorator

Question 4: Which design principle is best demonstrated by having an abstract `User` class with `Librarian` and `Member` as concrete subclasses?

- Encapsulation
- Aggregation
- Composition
- **[CORRECT]** Inheritance
  Explanation: `Librarian` and `Member` are specialized types of a `User`. They share common attributes and behaviors from the parent `User` class and also have their own specific functionalities, which is a classic example of inheritance.

Question 5: Which criterion is *not* supported by the `Search` interface?

- **[CORRECT]** ISBN
  Explanation: The `Search` interface defined in the case study specifies methods for searching by title, author, subject, and publication date, but not by ISBN.
- Title
- Author
- Publication date

12.4 Related case studies

Explore these related case studies to deepen your understanding of the design principles applied in the library management system:

- **Designing a car rental system:** This system also involves managing a catalog of rentable items (cars), handling user accounts, and processing reservations, checkouts, and returns, which closely mirrors the library's core logic.
- **Designing a hotel management system:** This case study requires designing a system for managing room bookings, guest information, and different staff roles, which involves similar reservation and user management principles.
- **Designing a video streaming service:** This involves managing a large digital catalog (movies, shows), handling user accounts with different subscription levels, and tracking user activity, relating to the catalog and user management aspects of the LMS.
- **Designing a university management system:** This system often includes a library module and deals with managing different roles, such as students and faculty (similar to members and librarians), course enrollments (similar to book reservations), and resource allocation.

- **Designing an e-commerce marketplace:** This involves managing a vast product catalog, seller and buyer accounts, handling orders, and processing payments and shipments, which is a more complex version of the inventory and transaction management found in the LMS.